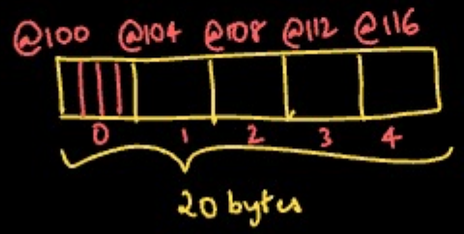
```
public interface List<T> {
    void add (T t);
    void insert (int i, T t);
    T get(int i);
    void set (int i, T t);
    T remove (int i);
    int size();
    boolean isEmpty();
    boolean remove (T t);
    void clear();
}
```

Dynamic Arrays

- ArrayList in java
  vector in C++
- Don't need to know size beforehand
- resizable

int - 4 bytes



Random access:

- accessing an element by its index
- very fast in arrays - $O(1)$

address of arr[i] = base addr + i * size of each element

addr of addr[3] = @100 + (3 * 4) = @112

0-3 - $\Theta(1)$
4 - $\Theta(n)$
5-7 - $\Theta(1)$
8 - $\Theta(n)$
9-15 - $\Theta(1)$
16 - $\Theta(n)$
17-31 - $\Theta(1)$
32 - $\Theta(n)$
33-63 - $\Theta(1)$

0	1	2	3	4
$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$

...

Total work for inserting 64 elements = $\frac{\quad}{64}$

Amortized running time for add at end = $\Theta(1)$

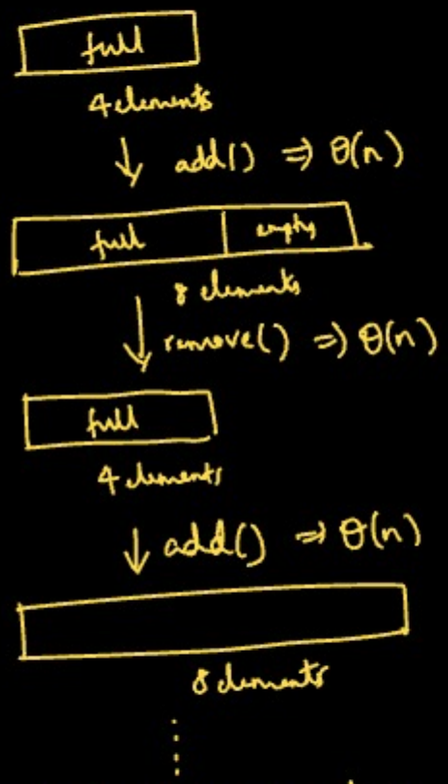
4 → 104 → 204 → 304 → ...

16 - capacity of the array

13 elements present

↓ remove() ops

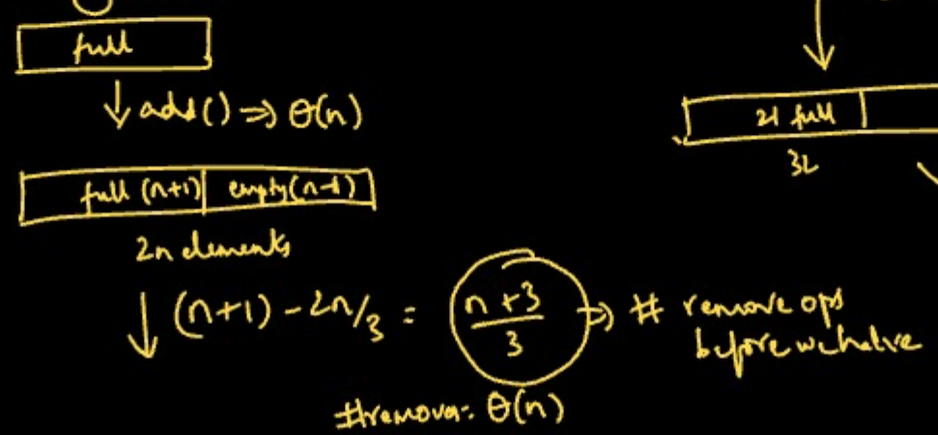
3 elements present

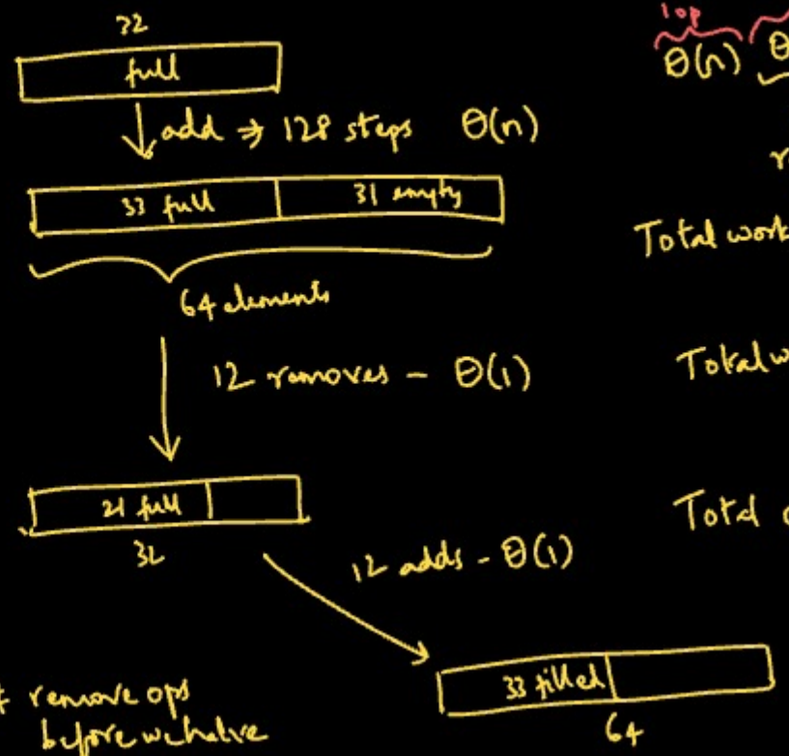


⋮

If we halve the array when #elements reach 1/2 capacity, alternating calls of add & remove can each be Θ(n)

to avoid this, we can halve the array once the #elements reach 1/3rd of the capacity instead of 1/2.





$\underbrace{\Theta(n)}_{\text{1 op}}\ \underbrace{\Theta(1)}_{\text{12 removes } \Theta(n)}\ \underbrace{\Theta(n)}_{\text{1 op}}\ \underbrace{\Theta(1)}_{\text{12 adds } (\frac{n+3}{3})}\ \Theta(n)$

Total work $= 3\Theta(n) + \underbrace{\Theta(n).\Theta(1)} + \underbrace{\Theta(n).\Theta(1)}$

Total work: $3\Theta(n) + 2\Theta(n)$
$= 5\Theta(n)$
$= \Theta(n)$

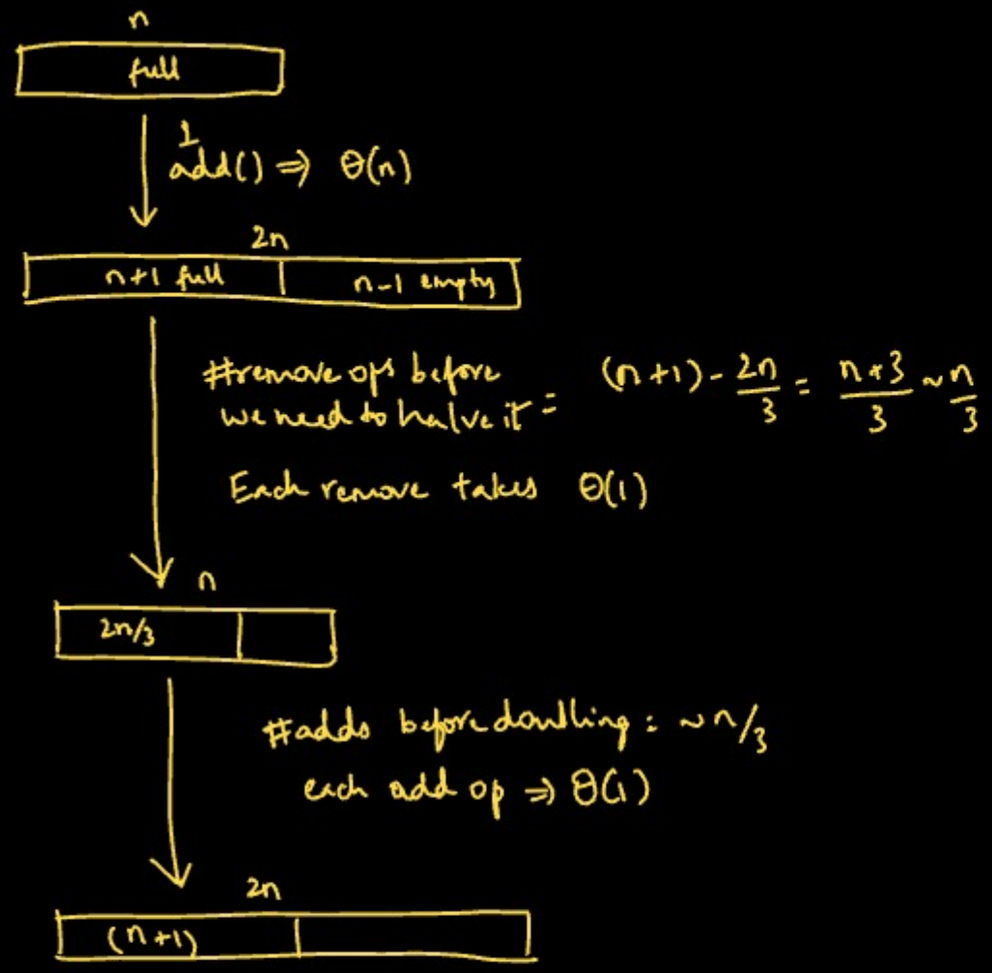
Total ops: $3 + 2\Theta(n)$

$\Theta(n)$ add, $\Theta(n)$ rem, $\Theta(n)$ add, $\Theta(n)$ remove ... n times

Total ops = $\Theta(n)$

Total work = $\Theta(n^2)$

Amortized RT = $\Theta(n)$ for each op



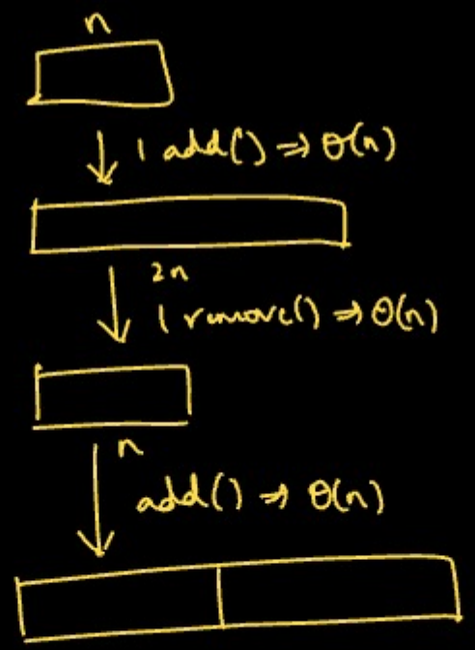
$$\Theta(3n^2)\cdot\Theta(n) = \Theta(3n^2\cdot n) = \Theta(3n^3) = \Theta(n^3)$$

$$\Theta(f(n))\,\Theta(g(n)) = \Theta(f(n)\cdot g(n))$$

Total work done: $\underbrace{\Theta(n)}_{\text{doubling}} + \underbrace{\frac{n}{3}\Theta(1)}_{\frac{n}{3}\text{ removes}} + \underbrace{\Theta(n)}_{\text{halving}} + \underbrace{\frac{n}{3}\Theta(1)}_{\frac{n}{3}\text{ adds}} + \underbrace{\Theta(n)}_{\text{doubling}} = 5\Theta(n) = \Theta(n)$

Total #ops: $\underbrace{1}_{\text{add()}} + \underbrace{\frac{n}{3}}_{\text{remove()}} + \underbrace{\frac{n}{3}}_{\text{add()}} = \Theta(n)$

Amortized RT per op $= \dfrac{\text{Total work}}{\text{Total ops}} = \dfrac{\Theta(n)}{n} = \Theta(1)$



Total work: $\overbrace{\Theta(n)}^{\text{add}} + \overbrace{\Theta(n)}^{\text{remove}} + \overbrace{\Theta(n)}^{\text{add}} = \Theta(n)$

#ops = 3

Amortized RT per op $= \dfrac{\Theta(n)}{3} = \Theta(n)$

```
int factorial (int n) {
    if (n ≤ 1) return 1;
    return n * factorial (n-1);
}
```

```
String reverse (String s) {          ABCD
    if (s.length() ≤ 1) return s;
    String lastChar = s[s.length()-1]; //D
    String prefix = s.substring(0, s.length()-1); //ABC
    return lastChar + reverse(prefix);
}
```

- Alg 1: $3n+5$ steps $= O(n)$, $O(n^2)$

  O- upperbound

  $1000n^2+10n = O(n)$ ✗, $O(n^2)$ ✓, $O(n^3)$ ✓

  $1000n^2+10n - O(n^2), \Omega(n^2) \Big\} \Theta(n^2)$

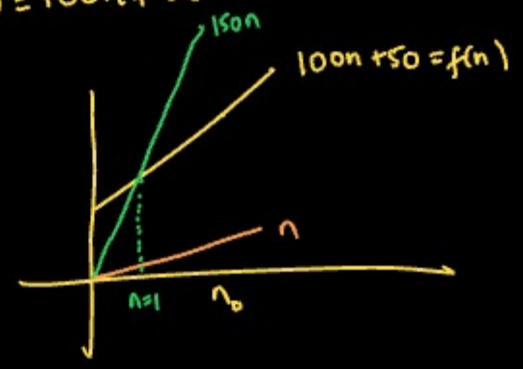
  Ω - lower bound

  $\underbrace{1000n^2+10n}$ = $\Omega(n)$ ✓, $\Omega(n^2)$ ✓, $\Omega(n^3)$ ✗

  $x \le 5$, $x \ge 5$ $\Rightarrow x=5$

  Linear search $= O(n)$ ✓, $O(n^2)$ ✓, $\Omega(n)$ ✓, $\Omega(1)$ ✓ $\Rightarrow \Theta(n)$

$f(n) = 100n + 50$



$\underbrace{100n+50}_{f(n)} \le \underbrace{150n}_{c=250}$ for all $n \ge 1$ $\Rightarrow n_0 = 1$

$\boxed{100n+50 = O(n)}$

$f(n) = O(g(n))$

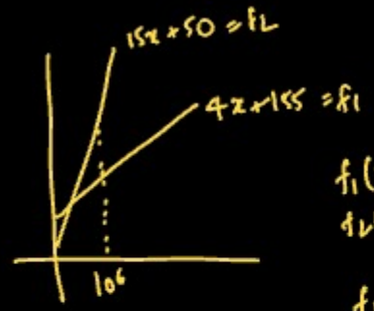
$\exists c, n_0$ s.t. $f(n) \le c \cdot g(n)$ $\forall n \ge n_0$

there exists — for all

$100n+50 \le 150n \ \forall n \ge 1 \Rightarrow 100n+50 = O(n)$

implies

$200n+50 \le 250n \ \forall n \ge 1 \Rightarrow 200n+50 = O(n)$



$f_1(1000) = 4155$
$f_2(1000) = 15050$

$f_1(1001) = 4159$
$f_2(1001) = 15065$

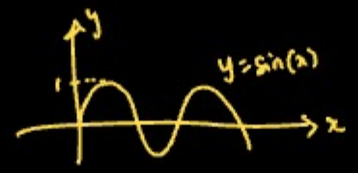
4

15

'n' - Linear in 'n' $\Rightarrow an+b$
- quadratic in 'n' $\Rightarrow an^2+bn+c$
- cubic an^3+bn^2+cn+d

$4x+15$
$15x+100$

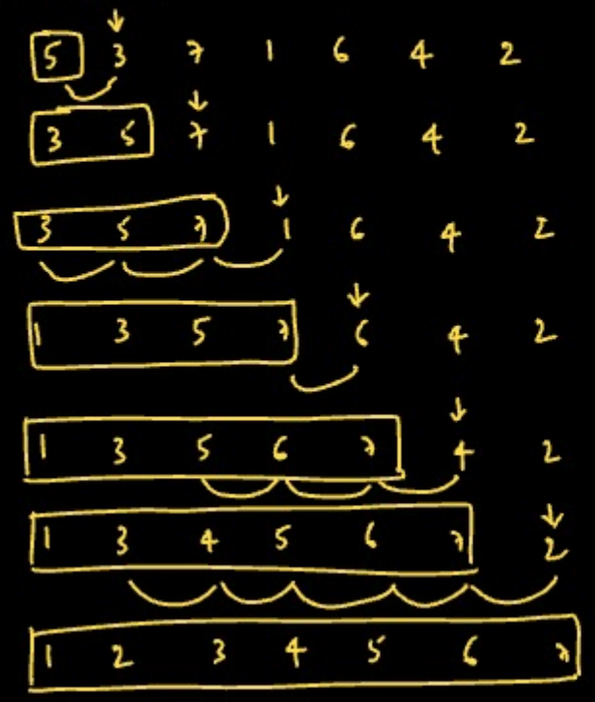
$4x^2+5x+10$
$9x^2+10x+15 = cx^2$



$10n+5 = O(n)$ ✓
$= O(n^2)$ ✓
$= O(1)$ ✗

$10n^2+5n+6 = O(n^2)$ ✓ $\quad \Omega(n^2)$ ✓ $\quad \Rightarrow \Theta(n^2)$
$O(n^3)$ ✓ $\quad \Omega(n^3)$ ✗
$O(n)$ ✗ $\quad \Omega(n)$ ✓
$O(n^{100})$ ✓

Insertion sort



	#comparisons	+	#swaps	RT
Best case	$n-1$		0	$\Theta(n)$
Worst case	$\frac{n(n-1)}{2}$		$\frac{n(n-1)}{2}$	$\Theta(n^2)$

Best case (already sorted)

#swap $= 0$

comparisons $= n-1$

1 2 3 4 5

(1,2)
(2,3)
(3,4)
(4,5)

Worst case (descending)

swaps =

$i=0$	0
$i=1$	1
$i=2$	2
$\vdots$	
$i=n-1$	$n-1$
	$\frac{n(n-1)}{2}$

comparisons:

$i=0$	0
$i=1$	1
$i=2$	2
$\vdots$	
$i=n-1$	$n-1$
	$\frac{n(n-1)}{2}$

RT of insertion sort is $O(n^2)$ ✓ $O(n)$ ✗
RT of insertion sort is $\Theta(n^2)$ ✗ $\Theta(n)$ ✗
RT of insertion sort is $\Omega(n^2)$ ✗ $\Omega(n)$ ✓

Best case RT of IS is $O(n^2)$ ✓ $O(n)$ ✓
Best case RT of IS is $\Theta(n^2)$ ✗ $\Theta(n)$ ✓
Best case RT of IS is $\Omega(n^2)$ ✗ $\Omega(n)$ ✓

Worst case RT of IS is $O(n^2)$ ✓ $O(n)$ ✗
" " " " " $\Theta(n^2)$ ✓ $\Theta(n)$ ✗
" " " " " $\Omega(n^2)$ ✓ $\Omega(n)$ ✓

Bubble sort

5 3 7 1 6 4 2

3 5 7 1 6 4 2

3 5 1 7 6 4 2

3 5 1 6 7 4 2

3 5 1 6 4 7 2

3 5 1 6 4 2 $\boxed{7}$

1 pass of Bubble sort

	#swaps	#Comparisons	RT
Best case	0	$n-1$	$\Theta(n)$
Worst case	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	$\Theta(n^2)$

Theoretically for comparison based sorting RT is $\Omega(n \log n)$ in the worst case.

- Merge sort

Selection sort

5 3 7 1 6 4 2 (i)

$\boxed{1}$ 3 7 5 6 4 2 (i)

$\boxed{1\ 2}$ 7 5 6 4 3 (i)

$\boxed{1\ 2\ 3}$ 5 6 4 7 (i)

$\boxed{1\ 2\ 3\ 4}$ 6 5 7 (i)

$\boxed{1\ 2\ 3\ 4\ 5}$ 6 7 (i)

$\boxed{1\ 2\ 3\ 4\ 5\ 6}$ 7 (i)

5 3 7 1 (i)

5 3 1 $\boxed{7}$ (i)

5 3 $\boxed{5\ 7}$ (i)

1 $\boxed{3\ 5\ 7}$ (i)

	#comparisons	#swaps	RT
Best case	$n(n-1)/2$	$n-1$	$\Theta(n^2)$
worst case	$n(n-1)/2$	$n-1$	$\Theta(n^2)$

Best case

#comparisons:

$i=0$	$n-1$
$i=1$	$n-2$
$i=2$	$n-3$
$\vdots$	$\vdots$
$i=n-2$	1
	$\frac{n(n-1)}{2}$

#swaps:

$i=0$	1
$i=1$	1
$i=2$	1
$\vdots$	
$i=n-2$	1
	$n-1$

```
class Student implements Comparable<Student>{
    public int compareTo(Student o){
        ...
        ...
    }
}
```

```
Student[] students = {...}
Arrays.sort(students);
```

123	801	801	123
479	181	403	147
801	812	812	181
812	452	123	403
452	123	147	452
147	403	452	479
181	147	479	801
403	479	181	812

LSD Radix sort
$\Theta(n)$

010 11 011

>> 4

00000101

1 cycle

4523

$\frac{4523}{100} = 45$

$45 \% 10 = 5$

$\underbrace{(x / 10^i)}_{30 \text{ cycles}} \% 10$

Stability

- Duplicate elements in the input array, the relative order if it's not disturbed, then the algorithm is stable
- If long distance exchanges exist, then it is not a stable sort

$A \Rightarrow B$

$\sim B \Rightarrow \sim A$ (Contrapositive)

long distance $\Rightarrow$ Not stable

stable $\Rightarrow$ not long distance

If it rains, ground gets wet $(A \Rightarrow B)$

A = rain

B = ground gets wet

If ground doesn't get wet, it didn't rain $(\sim B \Rightarrow \sim A)$

$\underbrace{A \Rightarrow B}$ $\underbrace{\sim B \Rightarrow \sim A}$

equivalent

A = is wet

B = rises in that direction

	Stable?
Selection sort	✗
Insertion sort	✓
Bubble sort	✓

A = long distance exch exist

B = Not stable

equivalent
- $A \Rightarrow B$ "If long distance exch exist, then it is not stable"
- $\sim B \Rightarrow \sim A$ "If it is stable, then long distance exch don't exist"

$B \Rightarrow A$ Converse

$\sim A \Rightarrow \sim B$ Inverse

} equivalent

A	B	$A \Rightarrow B$	$\sim B$	$\sim A$	$\sim B \Rightarrow \sim A$
T	T	T	F	F	T
T	F	F	T	F	F
F	T	T	F	T	T
F	F	T	T	T	T

Selection sort

- there are long distance exch
  ⇒ Not stable

23_A 5 6 11 23_B 2
↑i 9

2 5 6 11 23_B 23_A

Insertion sort:

5 6 11 23_A ↓ 23_B 2

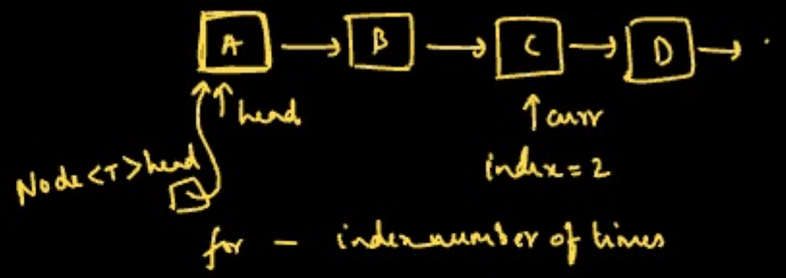
stable

Bubble sort

stable

Linked lists:

- Size can be modified after the list is created
- insertion at beginning is $\Theta(1)$ time
- Random access at a particular index in arrays – $\Theta(1)$
  But in linked lists – in the worst case it is $\Theta(n)$
- When we have the pointer to a linked list node, insert after that node is $\Theta(1)$

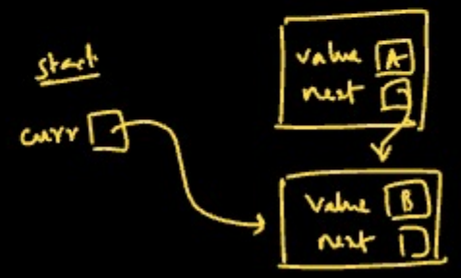


for – index number of times

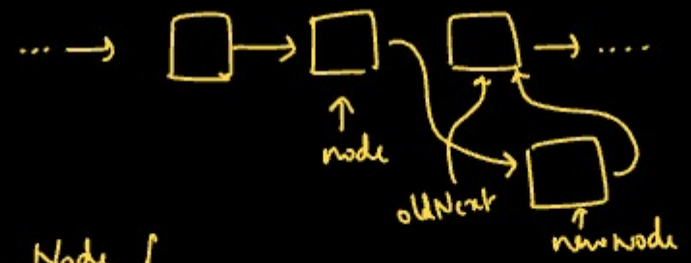
curr = curr.next

curr = curr.next

curr.value

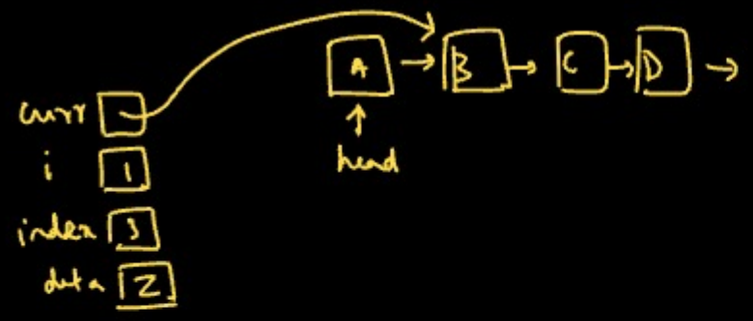


curr = curr.next




```
class Node {
    int data;
    Node next;
}
```

```
void insertAfter( Node node, Node newNode ){
    oldNext = node.next;
    node.next = newNode;
    newNode.next = oldNext;
}
```



- sentinel nodes
- insertLast in $\Theta(1)$
- deleteLast is still $\Theta(n)$

0 1 2 3

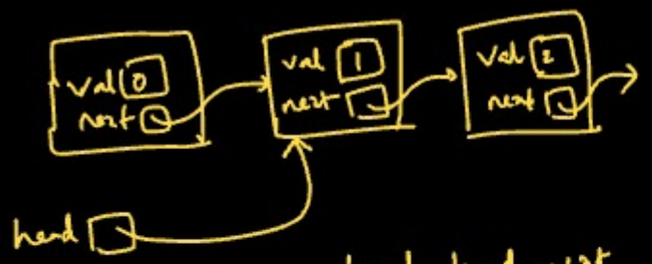
A → B → C → D →

ll.insert(2, "Z") A → B → Z → C → D →

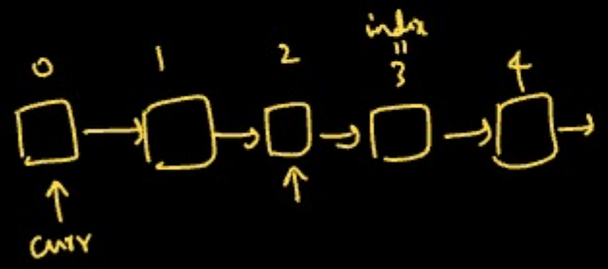
ll.insert(0, "a") a → A → B → Z → C → D →

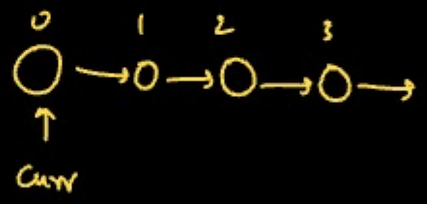
0 1 3 (size=3) 4 5

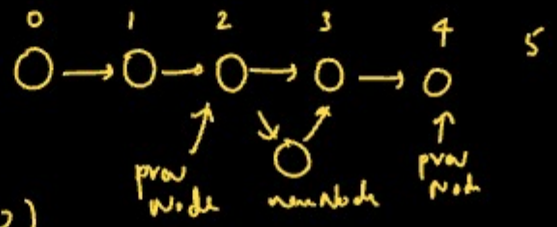
ll.insert(6, "k") a → A → B → Z → C → D → k →



head = head.next





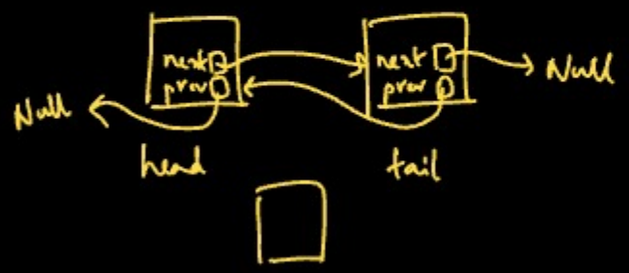


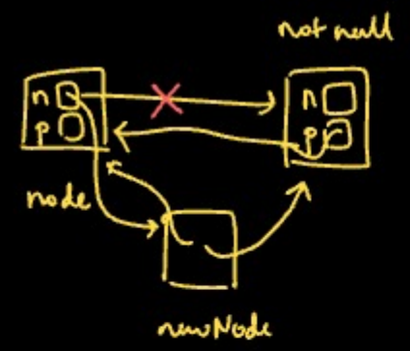
if (ind = 0)
  insertFirst(..)

prevNode = getNode(ind-1)

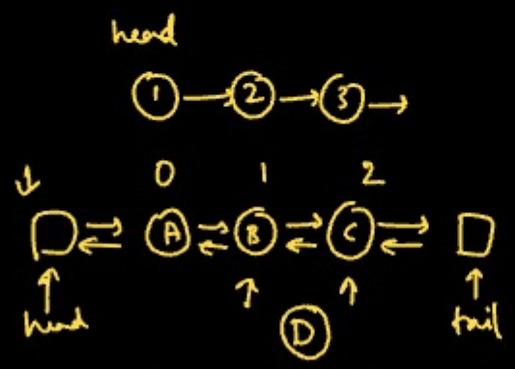
Sentinel nodes

- We always have two nodes (dummy) one before the list & one after the list

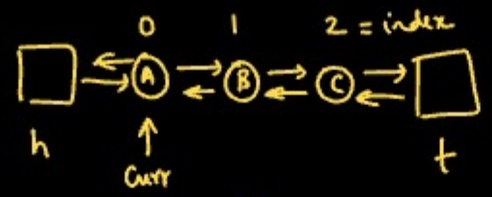




1 newNode.next = node.next
2 node.next = newNode
3 newNode.prev = node
4 newNode.next.prev = newNode



index = 2 → to Insert



get(1) → B

this loop runs 'n' times

```
for (i=0; i<n; i++) {
    ...
}
```

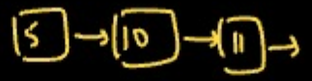
node.next.prev = node.prev

Stacks

- LIFO
- push()
  pop()
  top()

Queue

- FIFO
- enqueue()
  dequeue()
  peek()



Queue

5

Double ended Queue

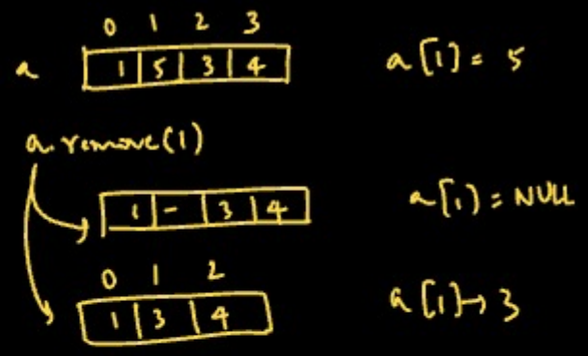
- Deque
- addFirst() addLast()
  removeFirst() removeLast()
  peekFirst() peekLast()

	Singly linked lists	Doubly linked lists
insertFirst	$\Theta(1)$	$\Theta(1)$
insertLast	$\Theta(1)$ with tail $O(n)$ without tail, $\Theta(n)$ worst case	$\Theta(1)$
deleteFirst	$\Theta(1)$	$\Theta(1)$
deleteLast	$\Theta(n)$ with/without tail	$\Theta(1)$

Dynamic Arrays

- We would increase the size (say make it 2x) whenever the array gets full

add(1)	$\Theta(1)$
add(5)	$\Theta(1)$
add(6)	$\Theta(1)$
add(8)	$\Theta(1)$
add(10)	$\Theta(n)$



Amortized running time

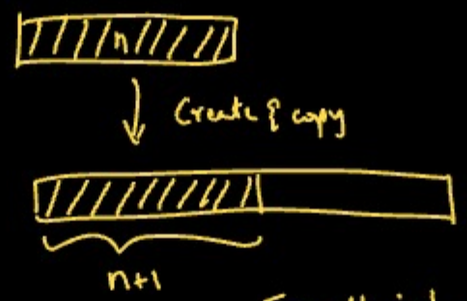
- Think of this as the avg running time per operation.
- Whenever we resize
  - $n \to kn$

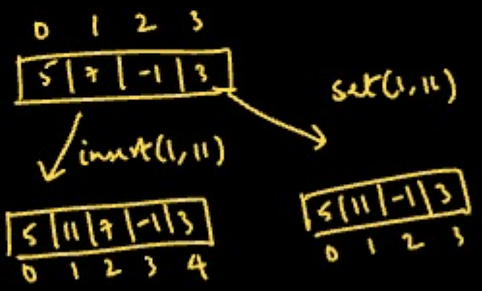
    $2n, 3n, 1.5n, 1.1n, \ldots$

    then add()'s amortized running time is $\Theta(1)$
  - $n \to n+100$

    then add()'s amortized running time is $\Theta(n)$

Idea: Have $\Theta(n)$ ops that take $\Theta(1)$ time between resizes.



For all 'n' new inserts = Total RT = $\underbrace{\Theta(n) + \Theta(1) \times n}_{n}$ = $\Theta(1)$

copy